Student Skill & Opportunity Tracker

Process, UI and backend development guide

Flask REST API · SQLite · vanilla JavaScript UI · unittest

1. Overview

This guide explains how the application is built: the architecture, the database, every API endpoint, the backend code, the web UI, the tests, and how to demonstrate it. The app lets a student keep skills, projects, certificates, learning goals and opportunities in one profile, see progress on a dashboard, and share a portfolio summary.

Development process followed

- Read the brief and turned the business rules into a checklist (ownership, URL validation, duplicates, future deadlines, overdue, status history, privacy).
- Designed the database first (section 3) and enforced rules there as well as in code.
- Built the REST API and tested each rule with automated tests.
- Built the UI last, as a thin layer that only calls the API.
- Added seed data, README and this guide.

2. Architecture

- Browser: one HTML page (`static/index.html`) with plain JavaScript. It never touches the database; it calls `/api/...` with `fetch`.
- Server: `app.py`, a Flask application created by `create_app()`. Flask serves the page, the JSON API and the public portfolio link.
- Database: SQLite file `tracker.db`, created from `schema.sql`.
- Login: a signed session cookie (HttpOnly, SameSite=Lax). Passwords are stored only as salted hashes.

Request flow: UI action, then `fetch` to the API, then `before_request` login check, then validation (`clean`), then ownership check (`owned`), then SQL, then JSON response, then the UI re-renders the view.

3. Database design

Table	Purpose	Key columns
users	login account	email (unique), pw_hash
profiles	1:1 with users	name, headline, bio, location, website, share_token
skills	student skills	user_id, name, level 1-5, tags
projects	projects	user_id, title, description, tech_stack, github_url, demo_url
certificates	certificates	user_id, title, issuer, issue_date, credential_url
goals	learning goals	user_id, title, target_date
milestones	checkable steps of a goal	goal_id, title, done 0/1
opportunities	internships etc.	user_id, title, organization, type, deadline, link, status
status_history	audit trail	opportunity_id, old_status, new_status, changed_at
activity	recent activity feed	user_id, text, at

Relationships: users 1-to-1 profiles; users 1-to-many skills, projects, certificates, goals, opportunities, activity; goals 1-to-many milestones; opportunities 1-to-many status_history. Foreign keys use `ON DELETE CASCADE`, so deleting a goal removes its milestones.

Integrity rules in the database itself:

- `CHECK(level BETWEEN 1 AND 5)`, `CHECK(type IN (...))`, `CHECK(status IN (...))`.

- Unique indexes on `(user_id, lower(name))` for skills and `(user_id, lower(title), lower(github_url))` for projects, so duplicates are impossible even if application code has a bug. Because `user_id` is part of the index, two different students may each have a skill called "Python".
- Percentages are never stored. Progress is calculated from milestones every time, so it can never become stale.

4. API reference

All endpoints return JSON. Errors look like `{"error": "Validation failed", "fields": {"github_url": "Enter a valid link..."}}`. Status codes: 200/201 success, 204 deleted, 401 not logged in, 404 not found or not yours, 409 duplicate, 422 validation failed.

Method and path	Purpose
POST /api/register, /api/login, /api/logout	account session
GET /api/me, PUT /api/profile	read / edit profile
GET, POST /api/skills; GET, PATCH, DELETE /api/skills/{id}	skills (?q= searches name and tags)
same pattern for /api/projects and /api/certificates	projects, certificates
GET, POST /api/goals; GET, PATCH, DELETE /api/goals/{id}	goals with milestones and progress
POST /api/goals/{id}/milestones	add milestone
PATCH, DELETE /api/milestones/{id}	check / uncheck ({"done": true}), rename, delete
GET, POST /api/opportunities; GET, PATCH, DELETE /api/opportunities/{id}	opportunities (?type=&status=&sort=&overdue=1&q=)
GET /api/opportunities/{id}/history	timestamped status changes
GET /api/dashboard	counts, upcoming, overdue, goal progress, activity
GET /api/portfolio, GET /api/portfolio/pdf	text summary, PDF download
POST /api/share, GET /public/{token}	create / revoke public link, public text view

Example: add a skill

```
POST /api/skills
{"name": "Python", "level": 4, "tags": "backend"}
201 {"id": 1, "name": "Python", "level": 4,
     "tags": "backend", "created_at": "2026-10-30T09:00:00+00:00"}
```

Example: validation error

```
POST /api/projects
{"title": "Demo", "github_url": "notaurl"}
422 {"error": "Validation failed",
     "fields": {"github_url": "Enter a valid link starting with http:// or https://"}}
```

Example: tick a milestone

```
PATCH /api/milestones/3   {"done": true}
200 {"id": 1, "title": "Learn SQL", "done": 1, "total": 4, "progress": 25, "milestones": [...]}
```

Example: change an opportunity status

```
PATCH /api/opportunities/2   {"status": "Applied"}
200 {"id": 2, "status": "Applied", "deadline": "2026-11-11", "overdue": false, "days_left": 12, ...}
```

5. Backend code explained (app.py)

Constants and `ApiError`. The allowed opportunity types and statuses live in tuples used by both validation and the UI. `OPEN_STATUS` lists the statuses that can become overdue. `SENSITIVE` is a regular expression that blocks text containing "password", "passport", "aadhaar", "ssn" or a 12-digit ID-like number. `ApiError` carries a message, an HTTP code and per-field errors; one error handler turns it into JSON, so route code can simply `raise`.

`clean(spec, data, partial)`**.** The single validation function. Each resource has a spec such as `"github_url": ("url", True, 300)`, meaning kind, required, max length. It trims text, enforces length, rejects sensitive text, checks URLs with `urlparse` (scheme must be http or https and a host must exist, so `javascript:` links are refused), parses dates with `date.fromisoformat`, checks levels and enumerations, and collects all field errors before raising one 422. `partial=True` is used for PATCH, where only the sent fields are checked.

`create_app()`**.** A factory so tests can create an app with a temporary database. It loads or generates a secret key (kept in an ignored file, so logins survive restarts), runs `schema.sql`, and defines all routes as inner functions that share helpers:

- `db()` opens one SQLite connection per request with foreign keys on and closes it in `teardown_appcontext`.
- `require_login` runs before every `/api/` call except register and login and returns 401 when the session has no user.
- `owned(table, id)` selects `WHERE id=? AND user_id=?`. If no row is found it raises 404, so another user's id is indistinguishable from a missing one. Every read, update and delete goes through it.
- `check_dup` compares lower-cased columns for the same user, excluding the row being edited, and raises 409.
- `log()` writes the activity feed entries shown on the dashboard.

Auth. `register` validates email, password length and name, stores `generate_password_hash(pw)`, creates the profile row and logs the user in. `login` returns the same error whether the email or the password is wrong, so it does not reveal which accounts exist.

`make_crud`**.** Skills, projects and certificates behave identically, so one function builds list, create, get, update and delete for each from a config dictionary. Column names come only from that config, never from user input, so the f-string SQL is safe; all values are passed as `?` parameters.

Goals and milestones. `goal_out` loads milestones and calculates `progress = round(100 * done / total)`, returning 0 when there are no milestones (no division by zero). Checking a milestone only writes if the value changes, which makes repeated clicks or retried requests idempotent. Milestone ownership is verified through a JOIN to the parent goal.

Opportunities. `opp_out` adds `days_left` and `overdue` (deadline before today and status still open). Creation rejects past deadlines; editing rejects only a changed deadline in the past, so old items can still be updated. In `opp_patch`, a status history row and an activity entry are written only when the new status differs from the current one. This is the rule for "repeated updates": sending Applied three times gives one history row. The list endpoint builds its SQL from an allow-list for sorting and `?` placeholders for filters.

Dashboard and portfolio. `dashboard` runs count queries and returns upcoming (next 30 days), overdue, goal progress and the last 10 activities. `portfolio_text` builds the summary from profile, skills, projects, certificates and goals only. The PDF endpoint feeds the same text to reportlab (escaped), so text and PDF always match. `share` creates a random `secrets.token_urlsafe` token; `/public/<token>` serves the text without login; setting the token to NULL revokes the link immediately.

6. Frontend explained (`static/index.html`)

- `api()` wraps `fetch`, sends the cookie, converts error responses into `Error` objects that carry `fields`, and shows the login screen on 401.

- `esc()` escapes every user-supplied value before it enters `innerHTML`, which prevents script injection (XSS). Links are also built with `rel="noopener noreferrer"`.
- `form()` is one reusable modal for every add/edit form. After a failed save it prints the server's message under each offending field, so URL and level errors appear exactly where the mistake is.
- `CFG` **and** `crudView()` describe skills, projects and certificates in data (fields and card template) and render list, search (debounced 250 ms), add, edit and delete from that. Each view has three states: loading text, an error box with a Retry button, and an empty-state message that tells a new student what to do first.
- `dashboard()` shows count tiles, upcoming deadlines, overdue items in red, progress bars and the activity feed; a new profile sees a welcome panel instead.
- `goals()` renders a progress bar and one checkbox per milestone; ticking calls the API and the view reloads, so the displayed percentage always comes from the server's calculation.
- `opps()` has type, status, sort and overdue-only controls, a status dropdown on each card, and a History button that shows the timestamped changes.
- `portfolio()` previews the shareable text, copies it, downloads the PDF, edits the profile and creates or revokes the share link.
- Layout uses CSS grid with `auto-fill`, so cards reflow from phone to desktop; colours use CSS variables with a dark-mode media query.

7. Testing (`tests/test_api.py`, 18 tests)

Each test builds a fresh temporary database. Covered: registration and login rules, 401 handling; skill validation and case-insensitive duplicates; URL validation including `javascript:` and `ftp:`; project duplicates; sensitive data rejection; goal progress at 0, 25, 50, 75, 100 percent, rounding (1 of 3 gives 33), empty goals and repeated checks; future-deadline rule; status history order and repeated updates; overdue flag (including Completed items not being overdue); filters and sorting; ownership (nine different cross-user requests all return 404, duplicates are per user); empty and populated dashboard; portfolio privacy, PDF output and share/revoke; persistence after restarting the app on the same database file.

Run: `python -m unittest discover -s tests -v`.

8. Demonstration script

1 `python seed.py`, `python app.py`, log in as demo@example.com. The dashboard shows counts, goal progress, upcoming and one overdue item.

2 Skills: add "Docker" level 3, then add "docker" again to see the duplicate error. Projects: add a project with GitHub link `notaurl` to see the URL error, then fix it.

3 Goals: create a goal with four milestones, tick them one by one and watch 25, 50, 75, 100 percent; untick one.

4 Opportunities: add one with a past date (rejected), add a valid one, change its status, open History. The seeded "Open Source Sprint" shows the Overdue badge; filter with Overdue only.

5 Portfolio: preview, download the PDF, create a share link and open it in a private window, then stop sharing.

6 Stop the server, start it again, log in: all data is still there.

9. Design decisions

- SQLite and Flask keep setup to one command and match the "local database or REST API" requirement.
- Rules are enforced in both the API and the database so that a bug in one layer cannot corrupt data.

- Progress and overdue status are computed, not stored, so they are always correct.
- Security: hashed passwords, ownership on every query, parameterised SQL, output escaping, http(s)-only links, no secrets in the repository.
- A "future deadline" is implemented as today or later, because an opportunity due today is still actionable.

10. Possible improvements

Password reset and email verification, rate limiting, CSRF tokens, pagination, file upload for certificates, drag-and-drop milestone ordering, and a charted progress history.